

```
# Organizational Godhard Mapping
## Cooperation, Coherence & Catastrophic Transitions in Teams
```

```
---
```

```
## Executive Summary
```

```
**Yes - and it's incredibly powerful.**
```

Organizations exhibit the same catastrophic dynamics as individuals, but with:

- **Multiple coupled Godhard curves** (one per person + emergent group-level curves)
- **Network contagion effects** (one team's collapse triggers others)
- **Coherence as a measurable physical quantity** (like phase coherence in physics)
- **Cooperation as potential energy minimization** (stable cooperation = deep potential well)

```
---
```

```
## Core Organizational Mappings
```

```
### Individual → Team → Organization Hierarchy
```

```
```python
```

```
"""
```

```
organizational_godhard_dynamics.py
```

```
Mapping Godhard curve dynamics to organizational cooperation and coherence
```

```
"""
```

```
from dataclasses import dataclass, field
from typing import Dict, List, Set, Tuple, Optional
import numpy as np
from enum import Enum
```

```
=====
ORGANIZATIONAL STATE PRIMITIVES
=====
```

```
class CooperationState(Enum):
 """Organizational cooperation regimes"""
 COLLABORATIVE = "collaborative" # Deep potential well, stable cooperation
 FUNCTIONAL = "functional" # Shallow well, cooperation maintained
 FRAGILE = "fragile" # Bistable, could flip to dysfunction
 SILOED = "siloeed" # Local minima, suboptimal equilibrium
 CONFLICTUAL = "conflictual" # Adversarial equilibrium
 DISINTEGRATING = "disintegrating" # No stable equilibrium
```

```
class CoherenceLevel(Enum):
 """Team coherence (alignment of individual states)"""
 HIGH_COHERENCE = "high" # All agents in phase (synchronized)
 MODERATE_COHERENCE = "moderate" # Some alignment
 LOW_COHERENCE = "low" # Chaotic, unsynchronized
 DECOHERENT = "decoherent" # Anti-correlated (canceling out)
```

```
@dataclass
```

```
class OrganizationalState:
```

```
 """
```

```
 Complete state of an organizational unit
```

Maps to both individual and collective Godhard curves  
"""

org\_id: str  
timestamp: datetime

# Individual states (from previous module)  
member\_states: Dict[str, 'GodhardAwareSystemState']

# EMERGENT COLLECTIVE PROPERTIES  
# These don't exist at individual level but emerge from interactions

# Cooperation metrics  
cooperation\_state: CooperationState = CooperationState.FUNCTIONAL  
cooperation\_potential\_depth: float = 5.0 # Depth of cooperation basin  
cooperation\_resilience: float = 5.0 # Barrier to defection

# Coherence metrics  
coherence\_level: CoherenceLevel = CoherenceLevel.MODERATE\_COHERENCE  
phase\_coherence: float = 0.5 # [0-1] How synchronized are individuals  
alignment\_score: float = 0.5 # [0-1] Goal/value alignment

# Collective Godhard properties  
collective\_curve\_position: float = 5.0  
collective\_bistability: bool = False # Can flip between cooperation/conflict  
collective\_distance\_to\_fold: float = 5.0

# Performance emergent from state  
productivity: float = 0.5  
innovation\_rate: float = 0.3  
decision\_velocity: float = 0.5

# Network topology effects  
network\_density: float = 0.5  
clustering\_coefficient: float = 0.5  
centralization: float = 0.5

# Critical transition indicators  
early\_warning\_signals: Dict[str, float] = field(default\_factory=dict)  
cascade\_vulnerability: float = 0.5

# =====  
# COOPERATION DYNAMICS (GAME THEORY + GODHARD)  
# =====

class CooperationPotential:

"""

Models cooperation as a potential energy landscape

Key insight: Cooperation is STABLE when it's a local minimum of potential  
Defection happens via catastrophic jumps over barriers

Based on:

- Evolutionary game theory (Prisoner's Dilemma, Public Goods)
- Catastrophe theory (Zeeman's model of aggression)
- Social physics (cooperation = order parameter)

"""

```

def __init__(self):
 # Game theory payoffs
 self.cooperation_benefit = 3.0 # Reward for mutual cooperation
 self.defection_temptation = 5.0 # Temptation to defect
 self.sucker_payoff = 0.0 # Payoff when cooperate but partner defects
 self.mutual_defection = 1.0 # Punishment for mutual defection

 # Godhard parameters
 self.trust_level = 0.5 # Affects barrier height
 self.reputation_cost = 2.0 # Cost of being known as defector

def potential_function(
 self,
 cooperation_level: float, # [0-1] fraction cooperating
 trust: float, # [0-1] trust between parties
 external_pressure: float # [0-10] stress on system
) -> float:
 """
 V(c, t, p) = potential energy as function of cooperation, trust, pressure

 Multiple minima create bistability:
 - Deep minimum at high cooperation (when trust high, pressure low)
 - Shallow minimum at low cooperation (defection equilibrium)
 - Unstable point in between (the fold)
 """
 c = cooperation_level
 t = trust
 p = external_pressure / 10.0 # normalize

 # Quartic potential (creates bistability)
 # $V(c) = c^4/4 - (\text{benefit} \cdot t) \cdot c^2/2 - (\text{payoff difference}) \cdot c + \text{pressure term}$

 benefit_term = self.cooperation_benefit * t
 pressure_term = p * (1 - c) # Pressure makes cooperation harder

 V = (
 0.25 * c**4 # Quartic term (stability at extremes)
 - 0.5 * benefit_term * c**2 # Cooperation benefit (creates minimum)
 - (self.cooperation_benefit - self.mutual_defection) * c # Payoff gradient
 + pressure_term # External pressure term
 + self.reputation_cost * (1 - c) * trust # Reputation cost for defection
)

 return V

def find_cooperation_equilibria(
 self,
 trust: float,
 pressure: float
) -> List[Tuple[float, str]]:
 """
 Find stable cooperation levels (equilibria)
 Returns: [(cooperation_level, stability_type), ...]
 """
 equilibria = []

```

```

Numerical search for $dV/dc = 0$
c_range = np.linspace(0, 1, 1000)
V = [self.potential_function(c, trust, pressure) for c in c_range]

Find local minima and maxima
for i in range(1, len(c_range) - 1):
 if V[i-1] > V[i] < V[i+1]: # Local minimum
 equilibria.append((c_range[i], "stable"))
 elif V[i-1] < V[i] > V[i+1]: # Local maximum
 equilibria.append((c_range[i], "unstable"))

return equilibria

def cooperation_basin_depth(
 self,
 cooperation_level: float,
 trust: float,
 pressure: float
) -> float:
 """
 How stable is current cooperation level?
 Deeper basin = more resilient cooperation
 """
 # Find current potential
 V_current = self.potential_function(cooperation_level, trust, pressure)

 # Find barrier to defection (saddle point)
 equilibria = self.find_cooperation_equilibria(trust, pressure)

 unstable_points = [c for c, stability in equilibria if stability == "unstable"]

 if not unstable_points:
 return 10.0 # Very stable

 # Find nearest saddle point
 nearest_saddle = min(unstable_points, key=lambda c: abs(c - cooperation_level))
 V_saddle = self.potential_function(nearest_saddle, trust, pressure)

 # Basin depth = barrier height
 basin_depth = V_saddle - V_current

 return max(0, basin_depth * 5) # Scale to [0-10]

def predict_cooperation_collapse(
 self,
 current_cooperation: float,
 trust: float,
 pressure: float,
 pressure_increase_rate: float
) -> Dict[str, float]:
 """
 Predict if/when cooperation will collapse as pressure increases

 Returns early warning of catastrophic defection cascade
 """
 # Simulate increasing pressure
 max_pressure = 10.0

```

```

pressure_trajectory = np.linspace(pressure, max_pressure, 100)

collapse_point = None

for p in pressure_trajectory:
 equilibria = self.find_cooperation_equilibria(trust, p)

 # Check if cooperation equilibrium disappears (fold catastrophe)
 high_coop_exists = any(
 c > 0.6 and stability == "stable"
 for c, stability in equilibria
)

 if not high_coop_exists and collapse_point is None:
 collapse_point = p
 break

if collapse_point:
 time_to_collapse = (collapse_point - pressure) / max(pressure_increase_rate, 0.01)

 return {
 'collapse_predicted': True,
 'collapse_pressure': collapse_point,
 'time_to_collapse': time_to_collapse,
 'warning_level': 1.0 if time_to_collapse < 5 else 0.5
 }
else:
 return {
 'collapse_predicted': False,
 'warning_level': 0.0
 }

```

```

=====
COHERENCE DYNAMICS (PHASE SYNCHRONIZATION)
=====

```

```

class CoherenceDynamics:
 """

```

Models team coherence as phase synchronization

Analogy: Like pendulums coupling together, people in teams can synchronize their states (mood, energy, focus) or decohere

Based on:

- Kuramoto model (phase oscillator synchronization)
- Quantum decoherence (information spreading into environment)
- Collective behavior physics

```

 """

```

```

def __init__(self):
 self.coupling_strength = 0.3 # How much people influence each other
 self.natural_frequency_variance = 0.5 # Individual differences

```

```

def compute_phase_coherence(
 self,
 individual_states: Dict[str, 'GodhardAwareSystemState']

```

```

) -> float:
"""
Compute Kuramoto order parameter: $R = |\langle e^{i\theta} \rangle|$

 $R = 1$: Perfect synchronization (high coherence)
 $R = 0$: Random phases (decoherence)

We map system state to phase: $\theta = 2\pi * (\text{strain} / 10)$
"""
if not individual_states:
 return 0.0

Convert states to phases
phases = []
for agent_id, state in individual_states.items():
 # Phase = position on cycle from calm to stressed
 theta = 2 * np.pi * (state.base_state.strain / 10.0)
 phases.append(theta)

phases = np.array(phases)

Kuramoto order parameter
$R = |\text{mean}(e^{i\theta})|$
complex_phases = np.exp(1j * phases)
mean_phase = np.mean(complex_phases)
R = np.abs(mean_phase)

return R

def compute_alignment_score(
 self,
 individual_states: Dict[str, 'GodhardAwareSystemState'],
 network: 'SocialNetwork'
) -> float:
"""
How aligned are goals/values across the team?

High alignment = people's trajectories point in same direction
Low alignment = people pulling in different directions
"""
if len(individual_states) < 2:
 return 1.0

agent_ids = list(individual_states.keys())

Construct "direction vectors" for each agent
Direction = gradient of their trajectory (where they're headed)
directions = {}

for agent_id, state in individual_states.items():
 # Direction vector in (pressure, margin, temperature) space
 # Positive components = increasing, negative = decreasing
 # This captures whether they're getting better or worse
 direction = np.array([
 -state.base_state.pressure, # Want pressure to decrease
 state.base_state.margin, # Want margin to increase
 -state.base_state.temperature # Want temperature to decrease
])

```

```

])
 directions[agent_id] = direction / (np.linalg.norm(direction) + 1e-10)

Compute pairwise alignment (dot products)
alignments = []
for i, aid1 in enumerate(agent_ids):
 for aid2 in agent_ids[i+1:]:
 # Dot product = cosine of angle between vectors
 alignment = np.dot(directions[aid1], directions[aid2])

 # Weight by connection strength if available
 if network:
 edge_data = network.graph.get_edge_data(aid1, aid2)
 weight = edge_data.get('strength', 1.0) if edge_data else 0.5
 alignment *= weight

 alignments.append(alignment)

Mean alignment
mean_alignment = np.mean(alignments) if alignments else 0.0

Map from [-1, 1] to [0, 1]
alignment_score = (mean_alignment + 1) / 2

return alignment_score

def predict_decoherence_cascade(
 self,
 initial_coherence: float,
 network: 'SocialNetwork',
 stressor_variance: float
) -> Dict[str, float]:
 """
 Predict if coherence will collapse (decoherence cascade)

 Like quantum decoherence: information leaks into environment,
 destroying correlations
 """
 # Decoherence rate = stressor_variance / coupling_strength
 # High variance + weak coupling → rapid decoherence

 decoherence_rate = stressor_variance / (self.coupling_strength + 0.1)

 # Coupling strength from network topology
 avg_degree = np.mean([
 network.graph.degree(node)
 for node in network.graph.nodes()
]) if network.graph.number_of_nodes() > 0 else 0

 effective_coupling = self.coupling_strength * avg_degree

 # Coherence decay: $dR/dt = -\gamma R + \kappa R(1-R)$
 # γ = decoherence rate
 # κ = recoupling rate

 if effective_coupling < decoherence_rate:
 # Decoherence dominates

```

```

time_to_decohere = -np.log(0.1 / max(initial_coherence, 0.01)) / decoherence_rate

return {
 'decoherence_predicted': True,
 'time_to_decoherence': time_to_decohere,
 'final_coherence': 0.1,
 'warning_level': 1.0 if time_to_decohere < 10 else 0.5
}
else:
 # Recoupling can maintain coherence
 steady_state_coherence = effective_coupling / (effective_coupling + decoherence_rate)

 return {
 'decoherence_predicted': False,
 'steady_state_coherence': steady_state_coherence,
 'warning_level': 0.0 if steady_state_coherence > 0.5 else 0.3
 }

def coherence_intervention_targets(
 self,
 individual_states: Dict[str, 'GodhardAwareSystemState'],
 network: 'SocialNetwork'
) -> List[Dict]:
 """
 Identify who to target to maximize coherence improvement

 Target highly central people (they couple many others)
 """
 if not network:
 return []

 centrality = network.compute_centrality_measures()

 # Find agents with high centrality but low state quality
 targets = []

 for agent_id, state in individual_states.items():
 if agent_id not in centrality:
 continue

 cent = centrality[agent_id]

 # High centrality + high strain = high-leverage target
 leverage = (
 0.4 * cent['degree'] +
 0.3 * cent['betweenness'] +
 0.3 * cent['eigenvector']
) * (state.base_state.strain / 10.0)

 targets.append({
 'agent_id': agent_id,
 'leverage': leverage,
 'centrality': cent,
 'current_strain': state.base_state.strain,
 'rationale': f"High centrality ({cent['degree']:.2f}) + high strain ({state.base_state.strain:.1f})"
 })

```

```

Sort by leverage
targets.sort(key=lambda x: x['leverage'], reverse=True)

return targets[:5] # Top 5 targets

=====
ORGANIZATIONAL STRUCTURE EFFECTS
=====

class OrganizationalStructure(Enum):
 """Common organizational topologies"""
 HIERARCHICAL = "hierarchical"
 FLAT = "flat"
 MATRIX = "matrix"
 NETWORK = "network"
 SILOED = "siloed"
 HYBRID = "hybrid"

@dataclass
class StructuralProperties:
 """
 Structural properties that affect Godhard dynamics
 """
 structure_type: OrganizationalStructure

 # Network topology metrics
 n_nodes: int
 n_edges: int
 density: float
 clustering: float
 avg_path_length: float

 # Hierarchy metrics
 hierarchy_depth: int # Levels in org chart
 span_of_control: float # Avg direct reports
 centralization_index: float # [0-1] Power concentration

 # Bottleneck identification
 bottleneck_nodes: List[str] # Single points of failure
 bridge_edges: List[Tuple[str, str]] # Edges whose removal disconnects graph

 def compute_structural_resilience(self) -> float:
 """
 How resilient is structure to node failures?
 Based on graph connectivity
 """
 # More paths = more resilience
 # Lower centralization = more resilience
 # Higher clustering = more resilience (local redundancy)

 resilience = (
 0.3 * self.density +
 0.3 * self.clustering +
 0.2 * (1 - self.centralization_index) +
 0.2 * (1 / max(self.avg_path_length, 1))
)

```

```

Penalty for bottlenecks
bottleneck_penalty = 0.1 * len(self.bottleneck_nodes) / max(self.n_nodes, 1)

return max(0, min(1, resilience - bottleneck_penalty))

def identify_cascade_amplifiers(self, network: 'SocialNetwork') -> List[str]:
 """
 Identify structural positions that amplify cascades

 These are agents whose failure triggers largest cascades
 """
 amplifiers = []

 # Betweenness centrality identifies bridges
 centrality = network.compute_centrality_measures()

 for agent_id, cent in centrality.items():
 # High betweenness = lies on many shortest paths
 # Their failure disconnects many pairs
 if cent['betweenness'] > 0.3: # Threshold
 amplifiers.append(agent_id)

 return amplifiers

class StructuralGodhardInteraction:
 """
 How organizational structure modulates Godhard dynamics
 """

 def __init__(self, structure: StructuralProperties):
 self.structure = structure

 def structure_amplifies_or_dampens(
 self,
 individual_catastrophe_risk: float,
 agent_id: str,
 network: 'SocialNetwork'
) -> Tuple[float, str]:
 """
 Does structure amplify or dampen individual catastrophe risk?

 Returns: (modified_risk, explanation)
 """
 centrality = network.compute_centrality_measures()

 if agent_id not in centrality:
 return individual_catastrophe_risk, "No network data"

 cent = centrality[agent_id]

 # Hierarchical structures amplify catastrophes at top
 if self.structure.structure_type == OrganizationalStructure.HIERARCHICAL:
 # High positions have more impact
 amplification = 1.0 + cent['eigenvector']

```

```

modified_risk = individual_catastrophe_risk * amplification
explanation = f"Hierarchical structure amplifies risk for high-status nodes ({amplification:.2f}x)"

Network structures distribute risk
elif self.structure.structure_type == OrganizationalStructure.NETWORK:
 # High degree provides support but also spreads contagion
 if cent['degree'] > 0.5:
 damping = 0.8 # Support effect
 amplification = 1.2 # Contagion spreading effect

 modified_risk = individual_catastrophe_risk * (damping + amplification) / 2
 explanation = "Network structure: high connectivity provides support but enables contagion"
 else:
 modified_risk = individual_catastrophe_risk * 0.9
 explanation = "Network structure: peripheral position partially protected"

Siloed structures contain but don't resolve
elif self.structure.structure_type == OrganizationalStructure.SILOED:
 # Catastrophes contained within silo
 damping = 0.7 # Won't spread widely
 persistence = 1.5 # But won't get external help

 modified_risk = individual_catastrophe_risk * persistence
 explanation = "Siloed structure: risk contained but persistent (no external support)"

else:
 modified_risk = individual_catastrophe_risk
 explanation = "Structure effect neutral"

return modified_risk, explanation

def optimal_intervention_topology(
 self,
 current_network: 'SocialNetwork'
) -> Dict[str, List[Tuple[str, str]]]:
 """
 Suggest structural changes to improve resilience

 Returns recommended edges to add/remove
 """
 recommendations = {
 'add_edges': [], # Add these connections
 'remove_edges': [], # Remove these connections
 'rewire_edges': [] # Change these connections
 }

 # Identify structural holes (missing connections that would help)
 centrality = current_network.compute Centrality Measures()

 # Find agents with high betweenness (bridges)
 bridges = [
 agent_id for agent_id, cent in centrality.items()
 if cent['betweenness'] > 0.4
]

 # Add redundant paths around bridges (reduce single points of failure)
 for bridge_id in bridges:

```

```

neighbors = current_network.get_neighbors(bridge_id)

Connect neighbors to each other (create bypass)
for i, n1 in enumerate(neighbors):
 for n2 in neighbors[i+1:]:
 # Check if edge already exists
 if not current_network.graph.has_edge(n1, n2):
 recommendations['add_edges'].append((n1, n2))

Identify isolated clusters that should connect
communities = current_network.detect_communities()
community_members = {}
for agent_id, comm_id in communities.items():
 if comm_id not in community_members:
 community_members[comm_id] = []
 community_members[comm_id].append(agent_id)

Add inter-community bridges
comm_ids = list(community_members.keys())
for i, c1 in enumerate(comm_ids):
 for c2 in comm_ids[i+1:]:
 # Find high-centrality member of each community
 c1_central = max(
 community_members[c1],
 key=lambda a: centrality.get(a, {}).get('degree', 0)
)
 c2_central = max(
 community_members[c2],
 key=lambda a: centrality.get(a, {}).get('degree', 0)
)

 recommendations['add_edges'].append((c1_central, c2_central))

return recommendations

```

```

=====
COLLECTIVE CATASTROPHE PREDICTION
=====

```

```

class CollectiveCatastrophePredictor:
 """

```

```

 Predict collective catastrophes (not just individual)

```

```

 Examples:

```

- Team dissolution
  - Department dysfunction
  - Organizational culture collapse
  - Mass resignation events
- ```

    """

```

```

    def __init__(
        self,
        cooperation_model: CooperationPotential,
        coherence_model: CoherenceDynamics
    ):
        self.cooperation = cooperation_model

```

```

self.coherence = coherence_model

def compute_collective_state(
    self,
    org_state: OrganizationalState,
    network: 'SocialNetwork'
) -> Dict[str, float]:
    """
    Compute collective Godhard curve metrics
    """
    # Average individual positions
    individual_positions = [
        state.curve_position
        for state in org_state.member_states.values()
    ]

    collective_position = np.mean(individual_positions) if individual_positions else 5.0

    # Collective bistability exists if enough individuals in critical zones
    n_in_critical = sum(
        1 for state in org_state.member_states.values()
        if state.in_critical_zone
    )

    collective_bistability = n_in_critical / max(len(org_state.member_states), 1) > 0.3

    # Collective resilience = minimum of structural and individual resilience
    individual_resiliences = [
        state.resilience
        for state in org_state.member_states.values()
    ]

    structural_resilience = 5.0 # Would compute from StructuralProperties
    individual_resilience = np.mean(individual_resiliences) if individual_resiliences else 5.0

    collective_resilience = min(structural_resilience, individual_resilience)

    # Coherence
    phase_coherence = self.coherence.compute_phase_coherence(org_state.member_states)
    alignment_score = self.coherence.compute_alignment_score(
        org_state.member_states, network
    )

    # Cooperation level (estimate from relationship states)
    cooperation_level = 0.7 # Would compute from actual interactions

    return {
        'collective_position': collective_position,
        'collective_bistability': collective_bistability,
        'collective_resilience': collective_resilience,
        'phase_coherence': phase_coherence,
        'alignment_score': alignment_score,
        'cooperation_level': cooperation_level
    }

def predict_collective_catastrophe_types(
    self,

```

```

org_state: OrganizationalState,
network: 'SocialNetwork'
) -> List[Dict]:
"""
Predict TYPES of catastrophes that could occur

Different from individual - these are emergent phenomena
"""
collective = self.compute_collective_state(org_state, network)
catastrophes = []

# 1. COOPERATION COLLAPSE
if collective['cooperation_level'] > 0.5 and collective['collective_bistability']:
    cooperation_prediction = self.cooperation.predict_cooperation_collapse(
        current_cooperation=collective['cooperation_level'],
        trust=collective['alignment_score'],
        pressure=collective['collective_position'],
        pressure_increase_rate=0.1 # Estimate
    )

    if cooperation_prediction['collapse_predicted']:
        catastrophes.append({
            'type': 'COOPERATION_COLLAPSE',
            'probability': 0.7,
            'time_horizon': cooperation_prediction['time_to_collapse'],
            'description': 'Team cooperation will flip from collaborative to adversarial',
            'warning_signs': [
                'Increasing defection in small decisions',
                'Blame language replacing collaborative language',
                'Information hoarding increases'
            ],
            'mechanism': 'Bistability: cooperation and defection both stable, system near fold'
        })

# 2. COHERENCE COLLAPSE (decoherence cascade)
if collective['phase_coherence'] > 0.4:
    # Estimate stressor variance
    individual_strains = [s.base_state.strain for s in org_state.member_states.values()]
    stressor_variance = np.var(individual_strains) if len(individual_strains) > 1 else 0

    decoherence_pred = self.coherence.predict_decoherence_cascade(
        initial_coherence=collective['phase_coherence'],
        network=network,
        stressor_variance=stressor_variance
    )

    if decoherence_pred['decoherence_predicted']:
        catastrophes.append({
            'type': 'COHERENCE_COLLAPSE',
            'probability': 0.6,
            'time_horizon': decoherence_pred['time_to_decoherence'],
            'description': 'Team will lose synchronization, become chaotic',
            'warning_signs': [
                'Meeting attendance becomes erratic',
                'Response times increase and vary widely',
                'Shared understanding breaks down'
            ],
        })

```

```
        'mechanism': 'Decoherence: coupling too weak to maintain phase coherence under noise'
    })
```

3. CASCADE RESIGNATION EVENT

```
n_near_fold = sum(
    1 for state in org_state.member_states.values()
    if state.distance_to_fold < 2.0
)

if n_near_fold / max(len(org_state.member_states), 1) > 0.4:
    catastrophes.append({
        'type': 'RESIGNATION_CASCADE',
        'probability': 0.5 + 0.3 * (n_near_fold / len(org_state.member_states)),
        'time_horizon': 5.0, # Relatively quick
        'description': 'Multiple resignations in short time window (cascade effect)',
        'warning_signs': [
            'Increased LinkedIn activity',
            'Decline in long-term commitments',
            'Withdrawal from optional activities'
        ],
        'mechanism': 'Contagion: one resignation lowers barriers for others'
    })
```

4. TRUST COLLAPSE

```
if collective['alignment_score'] > 0.4 and collective['collective_bistability']:
    catastrophes.append({
        'type': 'TRUST_COLLAPSE',
        'probability': 0.55,
        'time_horizon': 10.0,
        'description': 'Rapid loss of interpersonal trust across team',
        'warning_signs': [
            'Increased documentation/CYA behavior',
            'Escalations to management increase',
            'Side conversations replace direct communication'
        ],
        'mechanism': 'Hysteresis: trust builds slowly but collapses rapidly'
    })
```

5. SILO FORMATION

```
communities = network.detect_communities()
n_communities
```